

DSA4213 - LANGUAGE PROCESSING FOR DATA SCIENCE

NATIONAL UNIVERSITY OF SINGAPORE

DEPARTMENT OF STATISTICS AND DATA SCIENCE

Language Modelling and Topic Classification on AG News: Comparing Architectures and Embeddings

Author:
ZHANG BOXUAN (A0211187L)

Date: October 15, 2026

1 Introduction and Experiment Set-up

This project chose AG news dataset from Hugging face [1, 2]. The study aimed to compare different language models and choices of input-embedding on text prediction, and evaluated whether the representations support the given task of topic classification. This study firstly compare n-gram model, an RNN, an LSTM and a causal Transformer from the perspective of prediction quality, training cost, inference speed and generated text quality. Secondly, randomly initialized trainable embeddings, locally trained frozen CBOW embeddings and public frozen GloVe embeddings were compared. Lastly, document representations extracted from the random trainable and public-GloVe RNNs were compared for topic classification task.

The dataset contains four different topics: World, Sports, Business and Sci/Tech, inclduing existing split for training and test sets. Then the dataset has been balanced split into training subset of 20,000 articles and a validation subset of 2,000 articles from the training split, using seed 2026. The complete official test split contains 7,600 articles. The vocabulary was fitted only on training text and contains 10,000 entries, including padding, unknown, beginning-of-sequence and end-of-sequence tokens. Each article was processed as a separate sequence. Language models predicted tokens using only preceding context, and padding targets were excluded from the loss. Model selection used validation performance, followed by evaluation on the test split.

2 Part I: Comparison between Language Models

2.1 Comparison Setup

We compared an interpolated n-gram model [3], a unidirectional RNN [4], an LSTM [5] and a causal Transformer [6] using the same corpus, vocabulary and prediction targets. The n-gram model combined unigram, bigram and trigram probabilities. To select the optimal n-gram model with suitable smoothing and interpolation, several different options were compared. To make the task simpler, the best combination with the lowest perplexity score was selected for downstream analysis (Table 1). Additionally, unigram probabilities used add-alpha smoothing with $\alpha = 0.1$.

Table 1: Evaluation of interpolated n-gram models across different interpolation weights, cross-entropy (CE), and perplexity scores.

Model	**Weights** $(\lambda_1, \lambda_2, \lambda_3)$	**CE**	**Perplexity**
unigram	$(1.0, 0.0, 0.0)$	6.6781	794.83
bigram	$(0.1, 0.9, 0.0)$	5.4591	234.88
trigram_A	$(0.1, 0.3, 0.6)$	5.2075	182.64
trigram_B	$(0.2, 0.4, 0.4)$	**5.0906**	**162.48**
trigram_C	$(0.3, 0.5, 0.2)$	5.0960	163.37

All neural models used randomly initialized, trainable 100-dimensional embeddings. The RNN and LSTM each had one recurrent layer with 128 hidden units. The Transformer used two layers, four attention heads, a feedforward dimension of 256 and sinusoidal positional encodings. Neural models trained for five epochs using Adam, learning rate 0.001, batch size 32 and gradient clipping at 1.0. Checkpoints were selected using validation cross-entropy. The neural models were implemented using PyTorch 2.9.1 [7].

2.2 Training Cost, Model Size and Prediction Quality

Among the four comparisions, the n-gram model achieved the lowest test perplexity (162.59), while the RNN and Transformer performed similarly (Table 2). The LSTM had the highest perplexity under the shared five-epoch budget. The observed strong n-gram may be due to that recurring local phrases benefit count-based

prediction, while the neural models received a limited training budget in this context. In addition, since we used a small training set, the extra parameters in neural models become a liability. However, the small difference demonstrated does not establish a architectural advantage.

As for the training time required, n-gram counting took only 1.21 seconds. For neural models, equal epoch budgets was applied, therefore produced computational costs are measurable. Among the neural models, the LSTM trained fastest, followed by the RNN and Transformer. The Transformer had the fewest trainable parameters (2,194,912) and smallest strored size (8.382 MiB), yet the highest measured training time and inference latency. Therefore, parameter count did not directly correlate with the execution cost.

Table 2: Language-model prediction quality and measured computational costs.

Model	Test CE	Test PPL	Training (s)	Parameters	Stored size (MiB)
N-gram	5.091	162.59	1.21	Count tables	11.62
RNN	5.150	172.45	367.90	2,319,440	8.85
LSTM	5.218	184.54	185.39	2,407,760	9.19
Transformer	5.156	173.48	588.67	2,194,912	8.38

2.3 Inference Costs

Inference used the same 100 prefixes collected sequentially from the validation set, with each prefix capped at 64 tokens. Each request produced a full next-token probability distribution. Measurements followed one warm-up pass and used the median of three repetitions. Neural input tensors were prepared before timing, and device synchronization ensured completion of each request. Neural models recomputed the complete prefix without caching.

For the measured workload, the LSTM achieved the lowest latency at 0.93 ms per prefix, followed by the RNN (3.62 ms), n-gram model (6.75 ms), and Transformer (23.66 ms), with 1,079.5, 275.9, 148.2, and 42.3 predictions per second, respectively (Table 3). Each request computed a full next-token distribution, and neural models recomputed the complete prefix without caching. The n-gram ran on CPU, while the neural models used MPS. The above results were obtained by running on an Apple Silicon ARM64 device, and with limited generalization to other devices.

Table 3: Inference latency and throughput across models and hardware devices.

Model	Device	Latency (ms)	Predictions per second
N-gram	cpu	6.748	148.198
RNN	mps	3.624	275.950
LSTM	mps	0.926	1079.524
Transformer	mps	23.664	42.258

2.4 Learning Behavior

In this section, we set epoch = 5, validation cross-entropy decreased throughout all five epochs for all the neural models. The selected checkpoints were all from epoch 5, with validation cross-entropy of 5.157 for the RNN, 5.227 for the LSTM and 5.161 for the Transformer. At the final epoch, validation loss exceeded training loss by approximately 0.162, 0.039 and 0.266, respectively (Figure 1). The Transformer showed the largest gap, although its validation performance continued improving. The LSTM had both higher training and

validation loss. However, in the previous section, it showed to have the weakest prediction quaility. Therefore, a great overfitting from the curves may also directly lead to higher prediction quality.

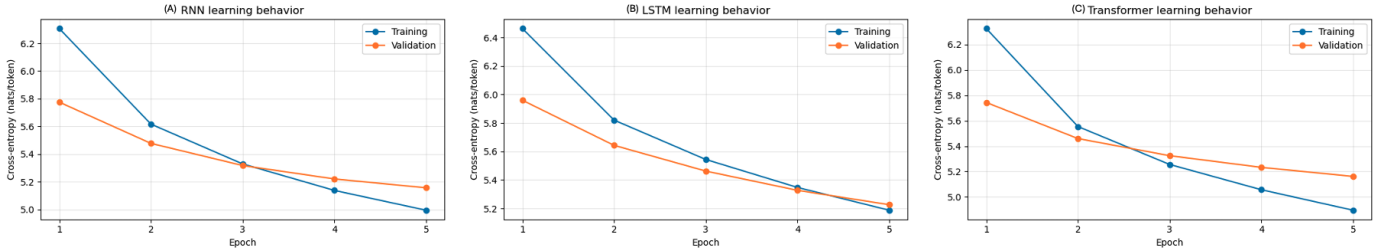


Figure 1: Training and validation cross-entropy curves across epochs for (A) RNN, (B) LSTM, and (C) Causal Transformer.

2.5 Generated-text Comparison

All models used the prompts “the government,” “the company” and “the team,” with top-k sampling (k=50), temperature 1.0 and a maximum of 40 new tokens for text generation (Table 4). Generation stopped earlier if an end-of-sequence token was sampled. The n-gram ended immediately for both “the government” and “the company,” despite achieving the lowest test perplexity. Neural models produced longer news-like fragments, but their continuations frequently contained unknown tokens, topic changes and grammatical errors. The LSTM’s continuation for “the company” shifted into baseball references and repeated “first inning.” The Transformer mixed political wording with sports references in its continuation for “the government.”

These examples show that useful local phrase patterns did not guarantee coherent text continuations. Perplexity evaluates next-token probabilities under observed contexts, whereas generation repeatedly conditions on the model’s own sampled tokens. A good local prediction quality does not ensure a coherent text generation. In summary, the n-gram offered the strongest prediction quality and lowest text generation ability under this setup. The LSTM offered the fastest measured neural execution, while the RNN achieved prediction quality close to the Transformer at lower measured cost. None of the models consistently generated coherent text.

Table 4: Generated-text comparison for the prompt “the government.

Model	Continuation excerpts	Main observation
N-gram	`. <eos>`	Ends immediately after punctuation, providing no substantive continuation.
RNN	`’ s top coach of <unk> -- based technology ... microsoft corp . on friday said ...`	Produces recognizable news fragments but combines coaching, technology and company references without a coherent sentence.
LSTM	`’ s top ceo <unk> <unk> a group ... the first second team ... a <unk> -- 1 win ...`	Shifts from corporate wording into sports references; frequent unknown tokens and broken phrasing limit readability.
Transformer	`’ s president michael owen ... the world cup teammates ... the yankees . <eos>`	Combines political wording with sports references, failing to maintain a consistent topic.

3 Part II: Embedding Comparison

3.1 Experimental Design

The following section compares the effects of using random/trainable baseline embeddings with randomly initialized, trainable input embeddings, locally trained frozen continuous bag-of-words (CBOW) embeddings

and public frozen GloVe embeddings. This section examined whether separately learned input embeddings provide useful information for language modeling. Based on the previous results, RNN was selected to investigate the effect of input embeddings. Its validation cross-entropy (5.150) was approximately equal to the Transformer's (5.156) and lower than LSTM (5.281), while its measured training time was shorter. All three settings retained the same 100-dimensional embeddings, one-layer RNN with 128 hidden units, vocabulary, data splits and language-model training procedure. Training used five epochs, Adam with learning rate 0.001, batch size 32 and gradient clipping at 1.0. We reuse the Part I RNN as the random/trainable embedding baseline. Two fresh RNNs used separately trained CBOW embeddings and public GloVe embeddings, respectively.

3.2 Embedding Preparation and Coverage

We trained our own embeddings using the CBOW approach [8], with only AG News training text. Each center token was predicted from the mean embeddings of up to two neighbouring tokens on each side. Contexts stayed within article boundaries, and padding was excluded from averaging. We transferred only the learned input-embedding table to the RNN. CBOW covered all 9,996 ordinary vocabulary entries.

Public embeddings used the 100-dimensional GloVe 6B release [9, 10], trained on Wikipedia 2014 and Gigaword 5, with 400,000 vocabulary entries. We obtained the vectors through Gensim's compressed distribution [11]. Public GloVe covered 9,841 entries, giving 98.45% vocabulary coverage and 99.63% frequency-weighted training-token coverage. These coverage calculations exclude the four special tokens. Missing GloVe words and non-padding special tokens received seeded random vectors drawn from N(0,1) and padding was zero. Matched GloVe vectors were retained without rescaling.

3.3 Prediction Quality and Training Cost

In table 5, among the three settings, frozen public GloVe performs the best with perplexity score of 156.99. It reduced the test perplexity by about 9% relative to the baseline. CBOW instead produced approximately 1.89% higher perplexity. It showed that full vocabulary coverage may not assure embeddings that improved next-token prediction.

Freezing the embedding table reduced trainable parammeters in both settings. Their measured RNN training times were much lower than the baseline required. If the separate CBOW training time is included, the combination time is 440.77 seconds. For the public GloVe, the excluding training time was not traced and recorded.

Table 5: RNN performance across input-embedding settings.

Setting	Best validation CE	Test PPL	LM training (s)	Separate local embedding training (s)	Trainable LM parameters
Random/trainable	5.157	172.45	367.90	—	2,319,440
Own CBOW/frozen	5.177	175.71	324.23	116.53	1,319,440
Public GloVe/frozen	5.063	156.99	247.32	—	1,319,440

3.4 Learning Behavior and Stability

CBOW had the lowest first-epoch validation cross-entropy of 5.634, compared with 5.776 for the baseline and 5.835 for GloVe (Figure 2). However, GloVe achieved the lowest validation loss from epoch 3 onward and finished with the strongest test performance. A better initial loss therefore did not show a better final model. Training and validation losses decreased throughout all three runs, with no recorded non-finite losses or validation-loss reversal. All settings selected the final epoch, so convergence was not demonstrated within the five-epoch budget.

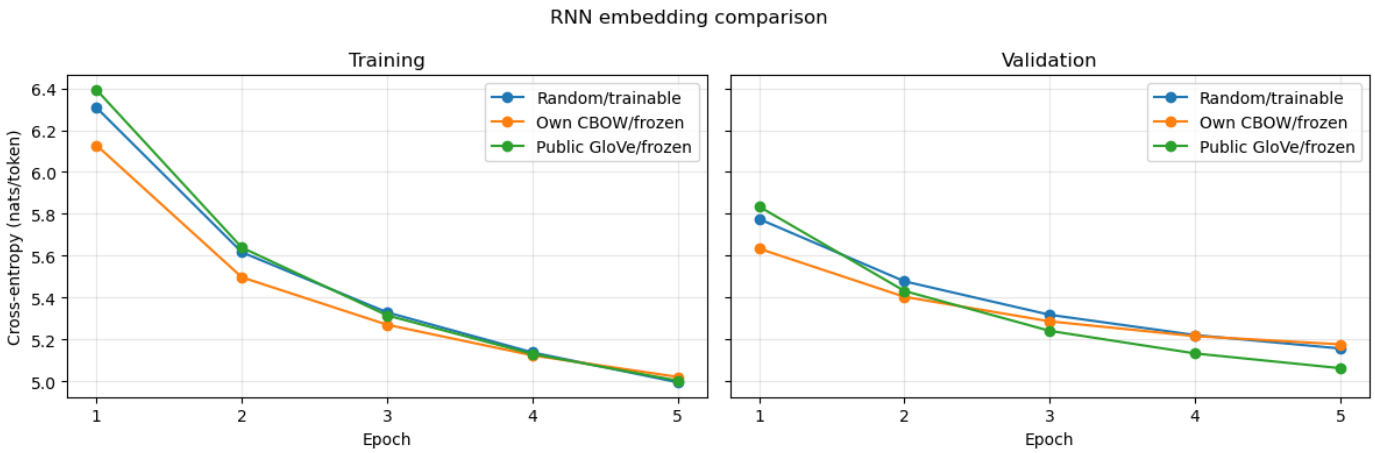


Figure 2: Training and validation cross-entropy for random/trainable RNN, CBOW and GloVE across epoches

3.5 Generated Text and Interpretation

Under the ”the sports” prompt, CBOW produced the clearest sports-related opening in this example, mentioning the Olympic Games and the national league. However, its full continuation subsequently became incoherent. The baseline generated six unknown tokens within a 14-token continuation, making its intended meaning difficult to recover. GloVe repeated “have” and ended its full continuation with repeated full stops. The other prompts also exposed failures. After “the company”, CBOW shifted from a web-browser reference into sports reporting, while GloVe repeated possessive fragments and introduced the American League.

Overall, a lower perplexity score did not establish superior generated-text quality. Perplexity measures probabilities under actual test contexts, however, text generation repeatedly uses the model’s own sampled tokens. These three matched prompts reveal specific failures but are insufficient to establish a general generation-quality ranking.

4 Part III: Downstream Representation Analysis

4.1 Representation Comparison

This experiment examines whether the language-model performance ranking from the previous section also holds for a supervised downstream task. AG News articles has been classified into World, Sports, Business and Sci/Tech using the same training, validation and test splits. Both trained RNN checkpoints were frozen. Each article became a 128-dimensional vector by averaging its contextual hidden states over text-token positions, excluding padding and boundary tokens. Both representations used a StandardScaler fitted only on training features, followed by logistic regression implemented in scikit-learn 1.6.1 [12]. We selected the regularization parameter C from $\{0.1, 1, 10\}$ using validation macro-F1. Both classifiers selected $C = 0.1$.

4.2 Classification Performance

With the give C=0.1, public GloVe improved accuracy by 19.26% and macro-F1 by 0.1931 compared to the random/trainable baseline (Table 6). These results indicate that the GloVe RNN’s hidden states contained more useful topic information for the chosen classifier. It might be due to GloVe was trained from more text, assiting the RNN to separate the news topic with complex texts.

As for the topic level confusion, GloVe improved recall for all four topics. Sports had the highest recall for both representations, increasing from 0.76 to 0.95. For the baseline, 0.19 of Business articles were classified as Sci/Tech, while 0.16 of Sci/Tech articles were classified as Business (Figure 3). GloVe reduced these rates to

Table 6: Held-out topic-classification results on 7,600 articles.

Representation	Selected C	Test accuracy	Test macro-F1
Random/trainable RNN	0.1	68.11%	0.6803
Public GloVe RNN	0.1	87.37%	0.8734

0.11 and 0.10, respectively. Additionally, the remaining wrongly-classified examples may due to cross-topic keywords or information overlaps that were difficult for the mdoel to distinguish.

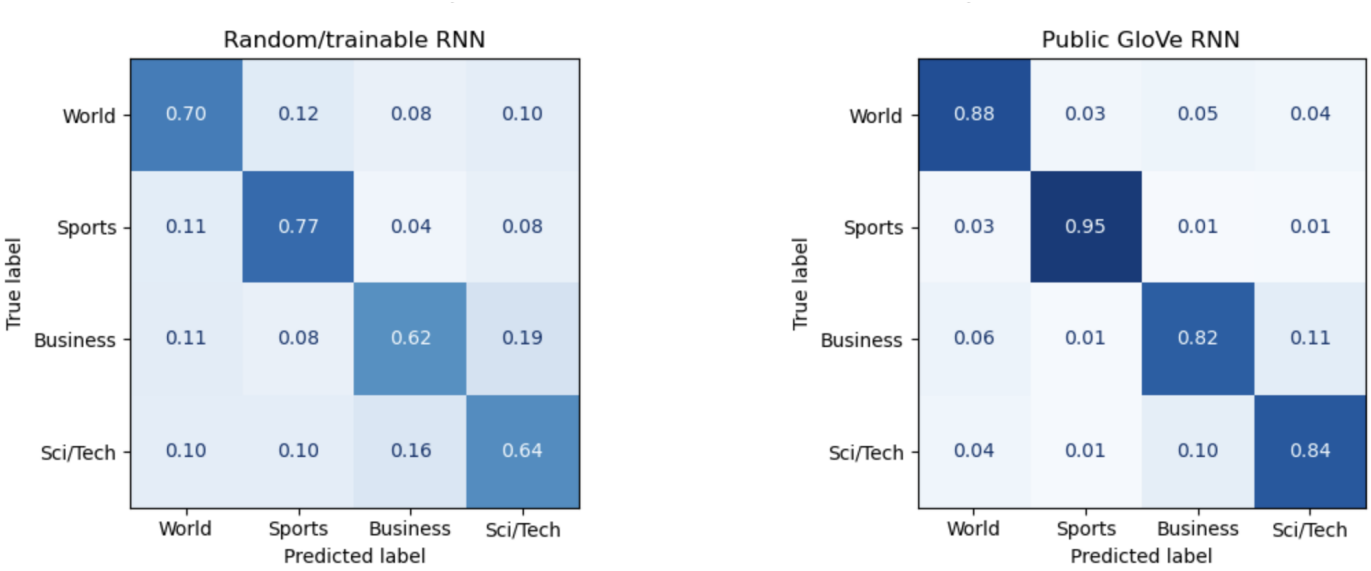


Figure 3: Row-normalized confusion matrices for the two representations.

4.3 Connection to Language-model Performance

In summary, the public-GloVe RNN performed better on both next-token prediction and topic classification. With the RNN architecture, pooling method and classifier procedure held constant, the downstream improvement indicates that the GloVe model's hidden states preserved more information useful for distinguishing news topics. However, the model is still struggling to generate coherent sentences with repetitive and grammatically weak texts as shown in the codebook.

5 Conclusions and Limitations

The experiments showed that architectural complexity might not directly correlate with better model performance under the selected training budget. The n-gram model achieved the lowest text perplexity in text prediction, while LSTM has the fastest neural execution. Public frozen GloVe improved RNN prediction quality, whereas locally trained frozen CBOW embeddings did not perform better than the baseline. However, neither pretrained setting consistently generated coherent text. Moreover, the GloVe RNN representation improved topic-classification accuracy, but errors still remained for cases with overlapping topics.

The main limitation in the study is the limited training setting, selected configurations and a five-epoch neural training budget. Longer training or different initializations may potentially change the rankings. Secondly, the embedding comparison did not consider the external training data from GloVe, thus, freezing may not be the only benefit for GloVe. Lastly, the context is specific to AG news classification tasl. The conclusion may be hard to generalize to other classification tasks.

References

[1] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, volume 28, 2015. https://arxiv.org/abs/1509.01626. pages 2

[2] fancyzhx. AG News dataset and dataset card. Hugging Face. https://huggingface.co/datasets/fancyzhx/ag_news. Accessed 13 October 2026. pages 2

[3] Daniel Jurafsky and James H. Martin. N-gram language models. Chapter 3 of Speech and Language Processing, third-edition draft, 2026. Draft dated 19 August 2026. https://web.stanford.edu/~jurafsky/slp3/3.pdf. pages 2

[4] Tomáš Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. Recurrent neural network based language model. In *Proceedings of INTERSPEECH*, 2010. https://www.isca-archive.org/interspeech_2010/mikolov10_interspeech.html. pages 2

[5] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. https://doi.org/10.1162/neco.1997.9.8.1735. pages 2

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. https://arxiv.org/abs/1706.03762. pages 2

[7] PyTorch contributors. PyTorch 2.9 documentation. PyTorch. https://docs.pytorch.org/docs/2.9/index.html. Experiments used PyTorch 2.9.1. Accessed 13 October 2026. pages 2

[8] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. arXiv:1301.3781, 2013. https://arxiv.org/abs/1301.3781. pages 5

[9] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1532–1543. Association for Computational Linguistics, 2014. https://aclanthology.org/D14-1162/. pages 5

[10] Stanford NLP Group. GloVe pretrained word vectors: Wikipedia 2014 and gigaword 5. Stanford University. GloVe 6B release, 100-dimensional vectors. https://nlp.stanford.edu/projects/glove/. Accessed 13 October 2026. pages 5

[11] Gensim-data contributors. glove-wiki-gigaword-100 pretrained-vector distribution. GitHub. https://github.com/piskvorky/gensim-data/releases/tag/glove-wiki-gigaword-100. Accessed 13 October 2026. pages 5

[12] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html. pages 6